

Data Science and Machine Learning for Geoscientists

Cheat sheet 2025/2026

Earth Science and Engineering, Imperial College London

Table of contents

1. Python Basics: Lists, Arrays, Functions	- 2 -
1.1 Lists (built-in Python)	- 2 -
1.2 List comprehensions	- 2 -
1.3 Tuples and dictionaries	- 2 -
1.4 Functions	- 3 -
1.5 Binary operators and comparisons	- 3 -
1.6 Python type() examples	- 3 -
1.7 for/while loops and if statements	- 3 -
2. NumPy: Arrays & Numerical Computing	- 4 -
2.1 Creating Arrays	- 4 -
2.2 Array Properties & Reshaping	- 5 -
2.3 Indexing & Slicing	- 5 -
2.4 Vectorized Operations	- 5 -
2.5 Aggregations & Statistics	- 6 -
3. Pandas: DataFrames & Series	- 6 -
3.1 Creating series and DataFrames	- 6 -
3.2 Inspecting Data	- 6 -
3.3 Selecting Data	- 6 -
3.4 Handling missing data	- 7 -
3.5 Adding, modifying, dropping columns	- 7 -
3.6 GroupBy and aggregation	- 7 -
3.7 Sorting	- 8 -
3.8 Statistics	- 8 -
3.9 Merging, joining, concatenating	- 8 -
3.10 Apply function	- 8 -
4. SciPy: Scientific Computing	- 8 -
4.1 Statistics (scipy.stats)	- 8 -
4.2 Optimization (scipy.optimize)	- 9 -
4.3 Linear Algebra (scipy.linalg)	- 9 -
5. Matplotlib: Plotting	- 9 -
5.1 Basic Line Plot	- 9 -
5.2 Scatter Plot	- 9 -
5.3 Bar Plot & Histogram	- 10 -
5.4 Subplots with histograms	- 10 -
5.5 Plot settings	- 10 -
6. Seaborn: Statistical Visualization	- 10 -
6.1 Basic Seaborn Plots	- 10 -
6.2 Distribution plots	- 11 -
6.3 Categorical & matrix plots	- 11 -
6.4 Style and themes	- 11 -
7. Scikit-Learn: Machine Learning Essentials	- 11 -
7.1 Train/test split	- 11 -
7.2 Feature scaling	- 11 -

7.3 Linear regression (regression task)	- 12 -
7.4 Logistic regression (classification)	- 12 -
7.5 Linear Regression with SGD	- 12 -
7.6 Imputer and fit/predict	- 13 -
7.7 Generic estimator pattern	- 13 -
7.8 Pipelines	- 13 -
7.9 Example fit/transform/predict	- 14 -
7.8 Regression metrics	- 15 -
7.9 Scores	- 16 -
7.10 Decision tree regressor and classifier	- 16 -
7.11 Model fine tuning	- 17 -
7.12 Cross-validation (return multiple scores)	- 18 -
7.13 Support vector regressor (SVR)	- 19 -
7.14 Support vector classifier (SVC)	- 19 -
7.15 KNeighborsRegressor	- 19 -
7.16 KNeighborsClassifier	- 20 -
7.17 Principal component analysis (PCA)	- 20 -

1. Python basics: lists, arrays, functions

1.1 Lists (built-in Python)

```
my_list = [1, 2, 3, 4]           # Create a list
my_list.append(5)                # Add one element at the end
my_list.extend([6, 7])          # Add multiple elements
my_list.insert(0, 0)             # Insert at index 0
len(my_list)                     # Length of list
3 in my_list                     # Membership test, returns True/False
my_list.remove(3)                # Remove first occurrence of value 3
popped = my_list.pop()           # Remove and return last element
sorted_list = sorted(my_list)   # Return a sorted copy
my_list.sort()                  # Sort list
my_list.sort(reverse=True)       # Sort list in descending order
my_list.reverse()               # Reverse list in place
sum(my_list)                    # Sum of numeric elements
min(my_list), max(my_list)       # Minimum, maximum of list
list(zip(list1, list2, list3))  # Merge elements of two or more lists

# SLICING: list[start:stop:step]
# - start: index of first element to include (default: 0).
# - stop:  index to stop BEFORE (excluded; default: len(list)).
# - step:  how many indices to move each time (default: 1).
list[a:b]      # elements from index a up to (but not including) b.
list[:b]       # from start up to index b-1.
list[a:]       # from index a to the end.
list[::2]      # every second element.
list[-n:]      # last n elements.
list[::-1]     # list in reverse order.
```

1.2 List comprehensions

```
squares = [x**2 for x in range(10)]           # Basic comprehension
evens = [x for x in range(20) if x % 2 == 0]   # With condition
pairs = [(i, j) for i in range(3) for j in range(2)] # Nested loops
```

1.3 Tuples and dictionaries

```
t = (1, 2, 3)           # Tuple (immutable list)
a, b, c = t              # Unpack tuple
```

```
d = {"a": 1, "b": 2}           # Dictionary (key-value store)
d["c"] = 3                     # Add new key
d.get("d", 0)                  # Get with default if missing
d.keys(), d.values(), d.items() # Views of keys, values, items
```

1.4 Functions

```
def add(a, b):
    return a + b                # Simple function

def func_with_args(*args, **kwargs):
    # *args: variable positional args, **kwargs: keyword args
    return args, kwargs

square = lambda x: x**2        # Anonymous (lambda) function
```

1.5 Binary operators and comparisons

```
# --- ARITHMETIC OPERATIONS ---
print(5 - 7)                    # 5 minus 7          -> -2
print(12 + 21.5)                # 12 plus 21.5        -> 33.5
print(12 / 4)                   # 12 divided by 4     -> 3.0
print(3 * 4)                    # 3 times 4           -> 12
print(13 % 4)                   # Modulo: remainder of 13 / 4, 4 * 3 = 12 -> 13 - 12 = 1
print(2 ** 4)                   # Exponentiation: 2 to the power of 4 -> 16

# --- COMPARISON OPERATORS ---
# Comparisons always return True or False
print(5 <= 2)                   # Less than or equal to      -> False
print(5 < 2)                    # Less than                  -> False
print(5 >= 2)                   # Greater than or equal to   -> True
print(5 > 2)                    # Greater than               -> True
print(5 == 2)                  # Equal to                  -> False
print(5 == 10/2)               # 5 == 5.0                  -> True
print(5 != 10/2)               # Not equal to              -> False
```

1.6 Python type() examples

```
# type() tells you the data type of a value or variable

# Basic types
print(type(5))                  # <class 'int'>             -> integer
print(type(3.14))              # <class 'float'>          -> decimal number
print(type("hello"))           # <class 'str'>            -> string (text)
print(type(True))              # <class 'bool'>          -> boolean (True/False)

# Collections
print(type([1, 2, 3]))         # <class 'list'>           -> list
print(type((1, 2, 3)))         # <class 'tuple'>         -> tuple
print(type({"a": 1, "b": 2}))  # <class 'dict'>          -> dictionary
print(type({1, 2, 3}))         # <class 'set'>           -> set
```

1.7 for/while loops and if statements

```
# ----- FOR LOOPS -----
# Loop over items in a list
for item in [1, 2, 3]:
    print(item)                # 1, then 2, then 3

# Loop over numbers 0 to 4
for i in range(5):
    print(i)                   # 0,1,2,3,4
```

```
# ----- WHILE LOOPS -----

# Repeat while condition is True
count = 0
while count < 3:
    print(count)          # 0,1,2
    count += 1            # IMPORTANT: update variable

# break: exit loop early
n = 0
while True:               # Potentially infinite loop
    if n == 3:
        break            # Stop when n == 3
    print(n)
    n += 1

# continue: skip to next iteration
x = 0
while x < 5:
    x += 1
    if x == 3:
        continue        # Skip printing when x == 3
    print(x)             # 1,2,4,5

# ----- IF STATEMENTS -----

age = 18

# if / else
if age >= 18:
    print("Adult")
else:
    print("Minor")

# if / elif / else chain
score = 72
if score >= 90:
    print("A")
elif score >= 75:
    print("B")
elif score >= 60:
    print("C")
else:
    print("D or below")

# Inline (ternary) if expression
status = "pass" if score >= 60 else "fail"
```

2. NumPy: arrays and numerical computing

Import convention:

```
import numpy as np
```

2.1 Creating arrays

```
a = np.array([1, 2, 3])          # 1D array from list
b = np.array([[1, 2], [3, 4]])   # 2D array (matrix)
```

```

np.zeros((3, 3))           # 3x3 array of zeros
np.ones(5)                 # 1D array of ones
np.full((2, 2), 7)         # 2x2 array filled with 7

np.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
# - Create a 1D NumPy array with regularly spaced values, np.arange(start,
#   stop, step, dtype=None)

# Parameters:
# - start: first value (included). Default is 0 if omitted.
# - stop:  upper limit (excluded).
# - step:  difference between consecutive values (default: 1), negative
#         decreases value
# - dtype: optional data type (e.g. int, float).

np.linspace(0, 1, 5) # 5 evenly spaced numbers from start to stop (inclusive)
np.eye(3)            # 3x3 identity matrix
np.random.rand(2, 3) # Uniform random [0,1), shape (2,3)
np.random.randint(0, 10, size=(2,2)) # Random ints between 0-9, shape (2,2)
np.random.randint(0, 10, 5) # Random numbers between 0 and 5

```

2.2 Array properties and reshaping

```

a.shape           # Tuple of dimensions
a.ndim            # Number of dimensions
a.size            # Total number of elements
a.dtype           # Data type of elements

b.reshape(4,)     # Reshape to 1D with length 4
b.reshape(1, 4)  # Reshape to 1x4
b.flatten()       # Return a flattened copy
c = b.ravel()     # View flattened (no copy if possible)

a.astype(float)   # Convert data type

```

2.3 Indexing and slicing

```

a = np.array([10, 20, 30, 40, 50])
a[0], a[-1]      # First and last elements
a[1:4]           # Slice indices 1,2,3
a[::2]           # Step of 2 (every other element)

B = np.array([[1, 2, 3],
              [4, 5, 6]])
B[0, 1]          # Element at row 0, col 1
B[:, 1]          # All rows, column 1
B[0, :]          # Row 0, all columns
B[B > 3]         # Boolean indexing: elements > 3
idx = np.where(a > 20) # Returns indices where condition is True

```

2.4 Vectorised operations

```

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b            # Elementwise addition
a - b            # Elementwise subtraction
a * b            # Elementwise multiplication
a / b            # Elementwise division
a ** 2           # Elementwise exponent
np.add(a, b)     # Same as a + b

```

```

np.multiply(a, b)           # Same as a * b
np.sqrt(a)                 # Elementwise square root
np.exp(a), np.log(a)       # Exponential, natural log

```

2.5 Aggregations and statistics

```

# For array a
a.sum()                    # Sum of all elements
a.mean()                  # Mean
a.std(), a.var()          # Standard deviation, variance
a.min(), a.max()          # Min, max
a.argmin(), a.argmax()    # Indices of min, max

a.sum(axis=0)              # Sum along axis (e.g., columns)
a.sum(axis=1)              # Sum for each row
np.cumsum(a)               # Cumulative sum
np.cumprod(a)              # Cumulative product
np.diff(array, axis=1)     # Calculate the cumulative difference
for an array along the given axis, e.g. np.diff([1, 4, 9]) returns [3, 5]

```

3. Pandas: DataFrames and series

Import convention:

```
import pandas as pd
```

3.1 Creating series and DataFrames

```

s = pd.Series([1, 2, 3], name="col")    # 1D labeled array

data = {"A": [1, 2, 3],
        "B": [4, 5, 6]}
df = pd.DataFrame(data)                 # DataFrame from dict

pd.read_csv("file.csv")                 # Load CSV into DataFrame
pd.read_excel("file.xlsx")              # Load Excel file
df.to_csv("out.csv", index=False)       # Save to CSV without index

```

3.2 Inspecting data

```

df.head()                            # First 5 rows (default, otherwise
indicate number of rows, e.g. df.head(10))
df.tail(3)                           # Last 3 rows
df.info()                             # Summary (types, non-null counts)
df.describe()
# Statistical summary of numeric cols
df.shape                              # (rows, columns)
df.columns                            # Column names
df.index                              # Row index
df.dtypes                             # dtype of each column (Series)
df["col"].dtype                       # dtype of one column
df_copy=df.copy()                     # copy Data Frame

```

3.3 Selecting data

```

df["A"]                              # Single column (Series)
df[["A", "B"]]                        # Multiple columns (DataFrame)

df.loc[0, "A"]                        # Label-based access (row=0, col="A")

```

```

df.loc[0:2, ["A", "B"]] # Slices by labels

df.iloc[0, 1] # Position-based access (row 0, col 1)
df.iloc[0:3, 0:2] # Position slices

df[df["A"] > 1] # Boolean filtering
df[(df["A"] > 1) & (df["B"] < 6)] # Combined conditions (AND)
df.select_dtypes(include = np.number) # Select numeric data types only

df.sample(n=5) # Randomly sample 5 rows from dataframe
df.sample(frac=0.10) # Randomly sample 10% rows from dataframe
df.sample(n=5, random_state=42) # Reproducible sampling
df.sample(n=3, axis=1) # Sample columns instead of rows

df.duplicated(subset=None, keep=False) # Filter duplicate rows where None
can be a column label, and keep: {'first', 'last', 'False'} marks which
occurrence to keep
df_nodup = df[df.duplicated(keep=False)] # Filter only duplicate rows (based
on all columns)
df_nodup = df.drop_duplicates() # Drop duplicates, keep first occurrence
df.drop_duplicates(inplace=True) # Drop duplicates and overwrite dataframe
df_nodup = df.drop_duplicates(subset=["col1", "col2"], keep="first") # Drop
duplicates based on specific column(s)

num_cols = df.select_dtypes(include = 'number').columns # Identify the
columns containing numerical features

```

3.4 Handling missing data

```

df.isna() # DataFrame of booleans (True if NaN)
df.isna().sum() # Count missing values per column
df.isnull().sum() # Count missing values per column
df.isnull().sum().sum() # Count total missing values
df[df.isnull().any(axis=1)] # Filter rows with any missing values
df[~df.isnull().any(axis=1)] # Filter rows with no missing values
df.dropna() # Drop rows with any NaN
df.dropna(subset=["A"]) # Drop rows where column A is NaN

df.fillna(0) # Replace NaN with 0
df["A"].fillna(df["A"].mean(), inplace=True) # Fill with column mean

```

3.5 Adding, modifying, dropping columns

```

df["C"] = df["A"] + df["B"] # New column from existing ones
df["log_A"] = np.log(df["A"]) # Use NumPy functions

df.rename(columns={"A": "feature_A"}, inplace=True) # Rename column
df.drop(columns=["C"], inplace=True) # Drop column C

```

3.6 GroupBy and aggregation

```

df.groupby("category")["value"].mean() # Mean by group
df.groupby("category").agg({
    "value": ["mean", "sum"],
    "other": "max"
}) # Multiple aggregations

df["category"].value_counts() # Frequency count per category

```

3.7 Sorting

```
df_sorted = df.sort_index() # ascending row index
df_sorted = df.sort_index(ascending=False) # descending row index
df_sorted = df.sort_index(axis=1) # sort columns by name
df_sorted = df.sort_values("col1") # by one column
df_sorted = df.sort_values(["col1", "col2"]) # multi-column
df_sorted = df.sort_values("col1", ascending=False)
df_sorted = df.sort_values(by=0, axis=1) # using row index 0
```

3.8 Statistics

```
df.cumsum() # cumulative sum down rows per column
df.cumprod() # cumulative product
df.pct_change() # down rows, per column
df["A"].pct_change() # for a single Series A
df.mean() # mean per column
df.mean(axis=1) # mean per row
df.sum() # column-wise sum (default axis=0) → Series
df.sum(axis=1) # row-wise sum → Series
df["col"].sum() # sum of one column
df.corr() # column-wise correlation
df.cov() # column-wise covariance

df.hist(bins=20, figsize=(10, 8), column=["col1", "col2"]) # histogram for
dataframe for a number of bins and a selection of columns. Requires
matplotlib.pyplot as plt, and plt.show() after this statement
plt.show()
```

3.9 Merging, joining, concatenating

```
pd.concat([df1, df2], axis=0) # Stack rows (same columns)
pd.concat([df1, df2], axis=1) # Join side by side (by index)

pd.merge(df1, df2, on="id") # SQL-style inner join by "id"
pd.merge(df1, df2, on="id", how="left") # Left join: keep all rows from df1
```

3.10 Apply function

```
df.apply(func, axis=0 or 1) # applies function along an axis of DataFrame

# Example: create a dataframe where column A is 0 if value<53, 1 otherwise
y = data['A'].apply(lambda x: np.where(x < 53, 0, 1)) # Stack rows (same
columns)
```

4. SciPy: scientific computing

Import example:

```
from scipy import stats, optimize, linalg
import numpy as np
```

4.1 Statistics (scipy.stats)

```
data = np.array([1, 2, 2, 3, 4])

stats.mean(data) # Mean
stats.median(data) # Median
stats.mode(data, keepdims=True) # Mode and count
stats.variation(data) # Coefficient of variation
```

```
stats.norm.pdf(x, loc=0, scale=1)      # Normal distribution PDF at x
stats.norm.cdf(x, loc=0, scale=1)      # CDF of normal distribution
stats.norm.rvs(size=100)               # Random samples from normal

t_stat, p_val = stats.ttest_ind(x, y)  # Two-sample t-test
```

4.2 Optimisation (scipy.optimize)

```
def f(x):
    return (x - 3)**2

result = optimize.minimize(f, x0=0)    # Minimize f starting from x=0
result.x                                # Optimal x value
result.fun                              # Function value at optimum
```

4.3 Linear algebra (scipy.linalg)

```
A = np.array([[1, 2],
              [3, 4]])

linalg.det(A)                          # Determinant
linalg.inv(A)                          # Inverse of A
w, v = linalg.eig(A)                   # Eigenvalues, eigenvectors
```

5. Matplotlib: plotting

Import convention:

```
import matplotlib.pyplot as plt
import matplotlib as mpl
```

5.1 Basic line plot

```
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(6, 4))              # Create new figure with size
plt.plot(x, y, label="sin(x)", color="blue", linewidth=2)
plt.title("Sine Wave")                  # Title of plot
plt.xlabel("x")                         # X-axis label
plt.ylabel("sin(x)")                    # Y-axis label
plt.legend()                            # Show legend
plt.grid(True)                          # Add grid
plt.show()                              # Display figure

plt.plot(df['A'])                        # Plot magnitude for column A
```

5.2 Scatter plot

```
x = np.random.rand(50) # Note x could be a series from dataframe, df["A"]
y = np.random.rand(50) # Note y could be a series from dataframe, df["B"]
colors = np.random.rand(50)
sizes = 100 * np.random.rand(50)

plt.scatter(x, y, c=colors, s=sizes, alpha=0.7, cmap="viridis")
plt.colorbar(label="Color scale")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Scatter Plot")
plt.show()
```

5.3 Bar plot and histogram

```
categories = ["A", "B", "C"]
values = [10, 15, 7]

plt.bar(categories, values, color="orange")
plt.title("Bar Plot")
plt.ylabel("Values")
plt.show()

data = np.random.randn(1000)

plt.hist(data, bins=30, color="steelblue", alpha=0.8)
plt.title("Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

5.4 Subplots with histograms

```
fig, axes = plt.subplots(2, 2) # Create subplot with 2 rows and 2 columns for
series A, B, C and D (e.g. columns of data frames)
axes[0,0].hist(df["A"], bins=70, color='r', alpha=0.5)
axes[0,1].hist(df["B"], bins=70, color='b', alpha=0.5)
axes[1,0].hist(df["C"], bins=70, color='g', alpha=0.5)
axes[1,1].hist(df["D"], bins=70, color='k', alpha=0.5)

plt.plot()
plt.show()

plt.savefig('path', format='png', dpi=300); # Save image as png
```

5.5 Plot settings

```
mpl.rcParams['axes', labelsizes=14) # Change the label text size on the axis name
mpl.rcParams['xtick', labelsizes=12) # Change the tick text size on the x-axis
mpl.rcParams['ytick', labelsizes=12) # Change the tick text size on the y-axis
plt.grid() # Add grid to plot

plt.minorticks_on() # Add minor ticks to plot
plt.xlabel('A'); plt.ylabel('B') # Add horizontal and vertical labels
```

6. Seaborn: statistical visualization

Import convention:

```
import seaborn as sns
```

6.1 Basic seaborn plots

```
sns.lineplot(data=df, x="A", y="B") # Line plot
plt.show()

sns.regplot(x="A", y="B", data=df) # Regression plot for series A and B of
dataframe df
plt.show()
```

```
sns.scatter(x="A", y="B", data=df) # Scatter plot for series A and B of
dataframe df
plt.show()
```

6.2 Distribution plots

```
sns.histplot(data=tips, x="total_bill", bins=20, kde=True)
plt.title("Histogram with KDE")
plt.show()
```

```
sns.boxplot(data=tips, x="day", y="total_bill")
plt.title("Boxplot by Day")
plt.show()
```

```
sns.violinplot(data=tips, x="day", y="total_bill")
plt.title("Violin Plot by Day")
plt.show()
```

6.3 Categorical and matrix plots

```
sns.countplot(data=tips, x="day")
plt.title("Count of Records by Day")
plt.show()
```

```
corr = tips.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

6.4 Style and themes

```
sns.set_style("whitegrid") # Styles: white, dark, ticks, etc.
sns.set_context("talk") # talk, paper, notebook, poster
```

7. Scikit-Learn: machine learning essentials

Import convention:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error,
max_error, r2_score
from sklearn.impute import SimpleImputer
```

7.1 Train/test split

```
train_set, test_set = train_test_split(data, train_size=0.7, random_state =
42) # This will give you 70% of the data in the training set, and 30% in the
test set, with a fixed random seed of 42 to be able to reduce the same split
across computers.
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7,
random_state = 42) # Similar to above, but split features (dropping target)
and target
```

7.2 Feature scaling

```
scaler = StandardScaler() # Standardize features (mean 0, std 1)
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test) # Scale values in X_test

=====

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1)) # Create a scaler that rescales
features to [0, 1] by default
scaler.fit(X_train) # Learn min and max for each feature from training data
X_train
X_train_scaled = scaler.transform(X_train) # Transform data: scale each
feature to the given range
X_test_scaled = scaler.transform(X_test) # use same scaler on test data
```

7.3 Linear regression (regression task)

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train) # Fit model to training data
y_pred = lin_reg.predict(X_test) # Predict for test data

mse = mean_squared_error(y_test, y_pred) # Mean Squared Error
rmse = mse**0.5 # Root Mean Squared Error
lin_reg.coef_, lin_reg.intercept_ # Model coefficients and intercept
```

7.4 Logistic regression (classification)

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(penalty = None, max_iter=1000) # No penalty
log_reg = LogisticRegression(penalty = 'l1', max_iter=1000, C=0.1) # L1
penalty, C is inverse of regularisation strength
log_reg = LogisticRegression(penalty = 'l2', max_iter=1000, C=0.1) # L2
penalty, C is inverse of regularisation strength

log_reg.fit(X, y) # Fit model
y_pred = log_reg.predict(X) # Predicted class labels
y_prob = log_reg.predict_proba(X)[:, 1] # Probabilities

coefficients = pd.DataFrame(logreg.coef_, columns = X.columns, index =
['Coefficients']) # Create a dataframe from the coefficients

coefficients = coefficients.T.sort_values(by = 'Coefficients', key = abs,
ascending = False) # Sorting coefficients

base_most_important = coefficients.head().index.to_list() # Extract the
index of the 5 rows with the highest coefficients
```

7.5 Linear regression with SGD

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter = 50, tol = 0, penalty = None, eta0 = 0.1,
random_state = 42) # 50 iterations, tolerance of 0 (not stopping based on
this), no penalty and training rate of 0.1

# SGD pipeline example:
sgd_pipe = Pipeline([('imputer', SimpleImputer()),
('scaler', MinMaxScaler()),
('regressor', sgd_reg)])
```

```
sgd_pipe.fit(X_train, y_train)
sgd_pipe['regressor'] # Extract regressor object
sgd_pipe['imputer'] # Extract imputer
```

7.6 Imputer and fit/predict

```
from sklearn.impute import SimpleImputer

# Create an imputer that replaces missing values with column means
imputer = SimpleImputer(strategy="mean")
# Learn imputation values (e.g., column means) from X
imputer.fit(X)

# Use learned values to fill missing entries in X
X_filled = imputer.transform(X)

# Fit + transform in one step (most common)
X_filled = imputer.fit_transform(X)
```

7.7 Generic estimator pattern

```
from sklearn.linear_model import LinearRegression

# 1) create model
model = LinearRegression()

# 2) fit: learn parameters from training data
model.fit(X_train, y_train)

# 3) predict: use model for new/unseen data
y_pred = model.predict(X_test)
```

7.8 Pipelines

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler

# Build a pipeline: sequence of steps applied in order (example)
# Steps:
# 1) imputer: fill missing values with column mean
# 2) scaler: scale features to [0, 1]
# 3) std_scaler: Standardization as scaling technique where the values in
the dataset are manipulated such that the scaled values are centred around a
mean of zero with a standard deviation of one
# Other frequent steps: 'logreg', LogisticRegression(); 'linreg',
LinearRegression()

my_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", MinMaxScaler(feature_range=(0, 1))),
    ("std_scaler", StandardScaler())]) # Standardization as scaling technique
where the values in the dataset are manipulated such that the scaled values
are centred around a mean of zero with a standard deviation of one

# Fit the whole pipeline on training data
# Internally:
# imputer.fit_transform(X_train)
# scaler.fit_transform(X_train_imputed)
```

```
# model.fit(X_train_scaled, y_train)
my_pipeline.fit(X_train, y_train)

y_test=my_pipeline.transform(y_test) # to transform test data

# Predict on new data (all steps applied automatically):
# imputer.transform(X_test)
# scaler.transform(X_test_imputed)
# model.predict(X_test_scaled)
y_pred = my_pipeline.predict(X_test)
```

7.9 Example fit/transform/predict

```
# =====
# CUSTOM HELPERS - fit_train_parameters & transform_data
# (COMMON PATTERN FOR PREPROCESSING WITH SKLEARN OBJECTS)
# =====

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler

# Suppose we want to:
# 1) Fit preprocessing steps on training data only
# 2) Reuse the fitted objects to transform any dataset (train / val /
test)

# Create preprocessing objects
imputer = SimpleImputer(strategy="mean")
scaler = MinMaxScaler(feature_range=(0, 1))

# -----
# 1) fit_train_parameters(X_train)
# -----
def fit_train_parameters(X_train):
    """
    Learn parameters (e.g., means, mins, maxes) from training data.
    This is a pattern name, not a built-in sklearn function.
    """
    imputer.fit(X_train) # learn imputation statistics
    X_train_imputed = imputer.transform(X_train)

    scaler.fit(X_train_imputed) # learn min, max for scaling
    X_train_scaled = scaler.transform(X_train_imputed)

    return X_train_scaled # preprocessed training data

# -----
# 2) transform_data(X)
# -----
def transform_data(X):
    """
    Apply preprocessing using parameters learned from training data.
    Again, this is a pattern name, not a built-in sklearn function.
    """
    X_imputed = imputer.transform(X) # use already-fitted imputer
    X_scaled = scaler.transform(X_imputed) # use already-fitted scaler
```

```

    return X_scaled

# =====
# USAGE EXAMPLE
# =====

# Fit preprocessing parameters on training data and transform it
X_train_prepared = fit_train_parameters(X_train)

# Transform validation/test data with the same parameters
X_val_prepared = transform_data(X_val)
X_test_prepared = transform_data(X_test)

# Then you can fit any model on the prepared data:
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train_prepared, y_train)
y_pred = model.predict(X_test_prepared)

```

7.8 Regression metrics

```

from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    max_error,
    r2_score)
import numpy as np

# y_true: true target values
# y_pred: predicted values from your model

# Mean Squared Error (MSE)
# Average of squared errors: mean((y_true - y_pred)^2)
mse = mean_squared_error(y_true, y_pred)

# Root Mean Squared Error (RMSE)
# Square root of MSE - same units as the target
rmse = mean_squared_error(y_true, y_pred, squared=False)
# or explicitly:
rmse_alt = np.sqrt(mse)

# Mean Absolute Error (MAE)
# Average of absolute errors: mean(|y_true - y_pred|)
mae = mean_absolute_error(y_true, y_pred)

# Max Error
# Worst (largest) absolute error among all samples
max_err = max_error(y_true, y_pred)

# R2 Score (Coefficient of Determination)
# 1.0 is perfect prediction, 0.0 ~ predicts like a constant mean model
r2 = r2_score(y_true, y_pred)

# =====
# TYPICAL WORKFLOW
# =====

# After training a regression model:
# model.fit(X_train, y_train)

```

```
# y_pred = model.predict(X_test)
# then compute:

mse = mean_squared_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
max_err = max_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

7.9 Scores

```
from sklearn.metrics import (
    precision_score,
    recall_score,
    accuracy_score,
    f1_score,
)

# y_true: true class labels
# y_pred: predicted class labels

acc = accuracy_score(y_true, y_pred) # Accuracy: fraction of correct
predictions: (TP + TN) / (TP + TN + FP + FN)
prec = precision_score(y_true, y_pred, average="binary") # Precision: Of all
predicted positives, how many are actually positive: TP / (TP + FP). For
multiclass/multilabel: average="macro", "micro", "weighted"
rec = recall_score(y_true, y_pred, average="binary") # Recall: of all real
positives, how many did we catch: TP / (TP + FN)
f1 = f1_score(y_true, y_pred, average="binary") # F1 score: Harmonic mean of
precision and recall
```

7.10 Decision tree regressor and classifier

```
from sklearn.tree import DecisionTreeRegressor

# DecisionTreeRegressor = tree-based model that predicts a continuous value
# by splitting the feature space into regions and assigning a constant
# prediction to each region (like piecewise-constant functions).

tree = DecisionTreeRegressor(
    criterion="squared_error", # how to measure split quality (formerly
    "mse")
    max_depth=None,           # limit tree depth (controls overfitting)
    min_samples_split=2,      # minimum samples to split an internal node
    min_samples_leaf=1,       # minimum samples at each leaf
    random_state=42
)

# Fit on training data
tree.fit(X_train, y_train)

# =====

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
# DecisionTreeClassifier = a tree-based classification model that
recursively
# splits the feature space using if-else rules to form "pure" regions and
# predicts class labels based on the majority class in each leaf node.
```

```

clf = DecisionTreeClassifier(
    criterion="gini",          # or "entropy", "log_loss"
    max_depth=None,           # limit tree depth to reduce overfitting
    min_samples_split=2,      # min samples to split an internal node
    min_samples_leaf=1,       # min samples at each leaf
    random_state=42
)

# Fit on training data
clf.fit(X_train, y_train)

# Plot tree
f, ax = plt.subplots(1,1, figsize = (45,15))
plot_tree(clf, max_depth = 4, feature_names = X_train.columns.to_list(), ax
= ax, fontsize = 16)
plt.show()

```

7.11 Model fine tuning

```

# -----
# LabelEncoder - encode labels
# -----
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_encoded = le.fit_transform(y)          # text labels -> ints
y_decoded = le.inverse_transform(y_encoded) # ints -> text labels

# -----
# RandomForestClassifier - tree ensemble classifier
# -----
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

# Or in a pipeline example:
pipe = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(random_state = 42))
])

# -----
# GridSearchCV - exhaustive grid search over params
# -----
from sklearn.model_selection import GridSearchCV

param_grid = {
    'scaler': [StandardScaler(), MinMaxScaler()],
    'model__max_depth': [3, 10, 30, 40],

```

```

    'model__n_estimators': [10, 100, 500]
}

grid = GridSearchCV(
    estimator=rf, # or pipe if defined
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
    verbose=5
)

grid.fit(X_train, y_train)
best_model = grid.best_estimator_ # Parameters of the best model

# Convert the results of the cross_validation to a dataframe, then sort
values to see the best performing model
pd.DataFrame(grid.cv_results_)[['params', 'mean_test_score', 'rank_test_score'
]].sort_values(by = 'rank_test_score')

# -----
# RandomizedSearchCV - random param search
# -----
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_grid = {
    'scaler': [StandardScaler(), MinMaxScaler()],
    'model__max_depth': randint(3, 40),
    'model__n_estimators': randint(10, 500)
}

rand_search = RandomizedSearchCV(
    estimator=rf, # or pipe if defined
    param_grid,
    n_iter=20,
    cv=5,
    scoring="accuracy",
    n_jobs=-1,
    verbose=5,
    random_state=42
)

rand_search.fit(X_train, y_train)
best_model rand = rand_search.best_estimator_ # Parameters of the best model

```

7.12 Cross-validation (return multiple scores)

```

from sklearn.model_selection import cross_validate
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42) # example for a
RandomForest, but could be logreg or any other

cv_results = cross_validate(
    estimator=model,
    X,                # features
    y,                # target
    cv=5,             # number of folds

```

```

        scoring="accuracy",    # or list/dict of metrics, e.g.
                               # 'neg_mean_squared_error'
        return_train_score=False
    )

# cv_results is a dict of arrays (one entry per fold)
# Common keys: 'test_score', 'fit_time', 'score_time'
mean_acc = cv_results["test_score"].mean()

```

7.13 Support vector regressor (SVR)

```

from sklearn.svm import SVR

svr = SVR(
    kernel="rbf",    # "rbf", "linear", "poly", "sigmoid"
    C=1.0,           # regularization strength (higher -> less
                    # regularization)
    epsilon=0.1,     # width of epsilon-insensitive tube
    gamma="scale"    # kernel coefficient for 'rbf', 'poly', 'sigmoid'
)

svr.fit(X_train, y_train)    # train model
y_pred = svr.predict(X_test) # predict continuous target

# Typical grid for search:
param_grid = {
    'svr__kernel': ["linear", "rbf", "sigmoid"],
    'svr__C': [0.1, 1, 10]
}
mean_squared_error

```

7.14 Support vector classifier (SVC)

```

from sklearn.svm import SVC

# SVC = Support Vector Machine classifier that finds a decision boundary
# (possibly non-linear via kernels) maximizing the margin between classes.

svc = SVC(
    kernel="rbf",    # "rbf", "linear", "poly", "sigmoid"
    C=1.0,           # regularization (higher C -> less regularization)
    gamma="scale",   # kernel coefficient for 'rbf', 'poly', 'sigmoid'
    probability=False # True if you need predict_proba (slower)
)

svc.fit(X_train, y_train)    # train classifier
y_pred = svc.predict(X_test) # predicted class labels

param_grid = {
    "C": [0.1, 1, 10],
    "gamma": ["scale", "auto"],
    "kernel": ["rbf", "linear"]
}

```

7.15 KNeighborsRegressor

```

from sklearn.neighbors import KNeighborsRegressor

# KNeighborsRegressor = predicts a value by averaging the target

```

```
# values of the k nearest training samples in feature space.

knn_reg = KNeighborsRegressor(
    n_neighbors=5,      # k: number of neighbors
    weights="uniform",  # "uniform" or "distance"
    metric="minkowski"  # distance metric; p=2 -> Euclidean
)

knn_reg.fit(X_train, y_train)    # store training data
y_pred = knn_reg.predict(X_test) # predict continuous target

# Example grid for search:
param_grid = {
    "n_neighbors": [3, 5, 7, 9],
    "weights": ["uniform", "distance"],
    "p": [1, 2]
}
```

7.16 KNeighborsClassifier

```
from sklearn.neighbors import KNeighborsClassifier

# KNeighborsClassifier = predicts the class of a sample by a majority vote
# among the k nearest training samples in feature space.

knn_clf = KNeighborsClassifier(
    n_neighbors=5,      # k: number of neighbors
    weights="uniform",  # "uniform" (equal) or "distance" (closer = more
                        # weight)
    metric="minkowski"  # distance metric; p=2 -> Euclidean
)

knn_clf.fit(X_train, y_train)    # store training data
y_pred = knn_clf.predict(X_test) # predicted class labels
y_proba = knn_clf.predict_proba(X_test) # class probabilities

# Example grid for search:
param_grid = {
    "n_neighbors": [3, 5, 7, 9],
    "weights": ["uniform", "distance"],
    "p": [1, 2]
}
```

7.17 Principal component analysis (PCA)

```
from sklearn.decomposition import PCA

# PCA = linear dimensionality-reduction method that finds orthogonal
# directions (principal components) capturing the maximum variance
# in the data and projects X onto a lower-dimensional space.

# Keep k principal components
pca = PCA(n_components=2)    # e.g., 2D for visualization

pca.fit(X_train)            # learn components from training data
X_train_pca = pca.transform(X_train)
X_test_pca = pca.transform(X_test)

# -----
# 3) EXPLAINED VARIANCE
# -----
```

```

explained_var = pca.explained_variance_ratio_ # fraction of variance per
component
total_explained = explained_var.sum() # total variance captured

# Choose components by variance target (e.g. 95%)
pca_95 = PCA(n_components=0.95) # keep enough components to explain 95%
variance
X_train_pca_95 = pca_95.fit_transform(X_train)

# -----
# 4) TYPICAL PATTERN IN A PIPELINE
# -----

pipe = Pipeline([
    ('pca', PCA()),
    ('knn', KNeighborsClassifier())
])

# Parameter grid
param_grid = {
    'pca__n_components': randint(low=10, high=20), # 10-20 n_components
    'knn__n_neighbors': randint(low=2, high=500)
}

# Random search using the appropriate values
knn_rnd_search = RandomizedSearchCV(
    knn_pipe,
    param_distributions = param_grid,
    n_iter=200,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42,
    verbose = 5
)

knn_rnd_search.fit(X.values, y) # Fit this model
y_pred = knn_rnd_search.predict(X_test) # Predict from this model

# Print best estimator and scores
print(knn_rnd_search.best_estimator_)
print(knn_rnd_search.best_score_)

```